

CS470 Homework 1 Report: FFJORD Implementation and Stability Analysis

20256443 Linda Shalash

October 26, 2025

Link to Google Colab Notebook and repository (for instructor's reference): https://colab.research.google.com/drive/1oCtWQJf_uVZW7TwjyqnhY1qPihTLE-HH?usp=sharing

1 Theoretical Background and Model Implementation

The **FFJORD (Free-form Jacobian of Reversible Dynamics)** model replaces a restrictive, discrete flow with a continuous flow defined by an Ordinary Differential Equation (ODE). This allows the mapping from the latent distribution $p_z(z)$ to the data distribution $p_x(x)$ to be performed by any neural network, $f(z(t), t)$. The change in log-likelihood is calculated via the continuity equation, which depends on the divergence of the dynamics:

$$\log p(z(t_1)) = \log p(z(t_0)) - \int_{t_0}^{t_1} \text{Tr} \left(\frac{\partial f(z(t), t)}{\partial z(t)} \right) dt$$

1.1 The Hutchinson Trace Estimator Implementation

To maintain $\mathcal{O}(D)$ complexity and allow unrestricted network architectures, the divergence $\text{Tr} \left(\frac{\partial f}{\partial z} \right)$ is estimated stochastically using the **Hutchinson Trace Estimator** with Rademacher probe vectors, $\epsilon \in \{\pm 1\}$. This method is essential because calculating the full Jacobian matrix ($\mathbf{J} = \frac{\partial f}{\partial \mathbf{z}}$), and subsequently its trace, costs $\mathcal{O}(D^2)$ time, making it infeasible for high-dimensional data like images. The estimator relies on the identity: $\text{Tr}(\mathbf{J}) = \mathbb{E}_{\mathbf{v}}[\mathbf{v}^\top \mathbf{J} \mathbf{v}]$.

The core of the implementation is the `trace_estimator_hutchinson` function, which leverages PyTorch's automatic differentiation capability for the Vector-Jacobian Product (VJP) without ever constructing $\mathbf{J}$ explicitly.

- **Function Signature:** The function signature confirms the key inputs: the dynamics f , the current state z (which requires gradient tracking), time t , and the number of probes K . K is averaged to reduce the variance of the stochastic estimate.
- **Probe Vector Generation (ϵ):** We draw the Rademacher vector $\mathbf{v}$ (which serves as the $\mathbf{v}$ in the identity) using `torch.randint(...)*2 - 1`. The Rademacher distribution is chosen over Gaussian because it typically exhibits lower variance for this specific trace estimation task.
- **VJP Calculation:** The crucial line `jvp = torch.autograd.grad(out, z, v, ...)[0]` computes the product $\mathbf{v}^\top \frac{\partial f}{\partial \mathbf{z}}$ (the Vector-Jacobian Product, or VJP). This step, done efficiently by PyTorch's autograd engine, provides the result of applying the Jacobian matrix to the probe vector $\mathbf{v}$.
- **Trace Estimate:** The final step, `trace_sum += (v * jvp).sum(dim=1)`, completes the Hutchinson identity $\mathbf{v}^\top \mathbf{J} \mathbf{v}$. The element-wise product of the probe vector ($\mathbf{v}$) and the resulting VJP ($\mathbf{J} \mathbf{v}$) followed by a sum ($\sum_i v_i (\mathbf{J} \mathbf{v})_i$) yields the final scalar estimate for the batch. This value is accumulated across K probes to yield the final, low-variance trace estimate.

2 Conclusive Analysis: Numerical Stability Bottleneck

2.1 Training Stabilization

Initial training using a fixed-step RK4 solver led to catastrophic collapse (Loss = 0.40). The model was successfully restarted with the **Adaptive DOPRI5 solver**. This solver detected the stiff, chaotic nature of the early learned dynamics, slowing training to ~ 17 seconds/batch to negotiate the instability.

The Epoch 2 parameters (Loss = 350.01) represent the last point of successful, stable training, offering the most reliable model state for generation.